

Structuring Meaning for Machine Cognition

Part I — Semantic Models, the Cognition Spectrum, and Shared References

Christopher Janz

Working paper. Part I of two.

Abstract

Network management is acquiring participants that reason. Autonomous systems now consume management information, act on it, and exchange it with one another. They bring machine cognition to a plane whose reasoning and design have until now lain with human engineers, and the information they are given was shaped for specific programs and the junctions between systems, not for comprehension. This paper argues that to serve such cognitive systems a representation must be made **complete**. It must be structured to carry complete meaning, so that a machine can comprehend it without a human supplying what a form leaves out. That completion is a **semantic model**, which for the purposes of this work is an ontology, including its lexicon, together with pragmatics, carried over a base of provenance. Producing one is what we call the **lift**, and the lift is what packages information for consumption by a machine cognitive system. **Reconciliation** is a distinct operation. It enters when two different semantic models must be made to correspond, for instance to bridge two modelling bases, and it is carried out at the level of the semantic models themselves. Where systems must then exchange native data, translators between their data models are generated as artefacts of the single reconciled model. The paper also introduces a *cognition spectrum*, which asks where the machine cognition sits relative to the systems being bridged. That question reframes the older one of how data is modelled, and it governs much of what follows. Against that background the paper develops the semantic model, the lift, and reconciliation. It sets out the shapes of divergence. It then examines shared references, the thin and practical anchors that reduce the work of reconciliation and, where machine cognition is scarce, the load on human supervision. Four worked demonstrations ground the argument, spanning configuration, intent, cross-domain provisioning, and observability. The paper closes with the goals of a proposed study, developed in Part II, to find effective shared references and to quantify their benefit across realistic cases and across the cognition spectrum.

1 Why machine cognition changes the problem

For most of its history, network management has moved information between programs. A controller renders a configuration, a device returns operational state, a collector gathers telemetry. The schemas that carry all of this, the data models, were designed so that software could produce and consume them reliably. The meaning of a field lived in a specification and in the minds of the engineers who wrote the software. The data model itself carried form, and left much of the meaning implicit. The reasoning that closed the gap, and the design that anticipated it, lay with human engineers.

That arrangement is now under pressure. The systems joining the management plane increasingly reason for themselves. They are asked to take an intent and decide how to satisfy it, to notice that something is wrong and work out what it means, and to negotiate with a peer that models the world differently. A machine cognitive consumer needs more

than a well-formed document. It needs to *comprehend* what the document means, including the parts a human engineer would have supplied from context: what a value is for, whose decision it reflects, whether it describes what was asked or what is actually happening, and how far it can be trusted.

Two consequences follow, and they organise this paper. The first is that information intended for a machine cognitive consumer should be made **complete**. It should be structured as meaning, not only as form, so that it can be comprehended without a human in the loop. We will call that completion a **semantic model**, and much of the paper says what it is, an ontology, its lexicon, and pragmatics, and how one is produced. The second consequence is that two systems usually hold different models. This is the normal case, not the exception, and they need a way to reconcile those models that does not depend on a universal model agreed in advance. The two needs are easily bundled together but are strictly distinct. One is **completing** a single model so that a machine can comprehend it at all. The other is **reconciling** two models so that machines which comprehend differently can meet. Machines can do the second between themselves, *ad hoc*. But how well they can do it, and how much help a human must give, depends on where the machine cognition sits. That question reframes everything else, and it is where the arrival of machine cognition most sharply departs from the data methodologies we have relied on. For that reason the paper takes it up first, in Section 2, before turning to how meaning should be structured. Part II develops a research programme around these questions. Part I sets out the theory and the terms.

These questions are being taken up, from different angles, across the network-management community, and the present work is meant to sit alongside those efforts rather than apart from them. Representing network operations as knowledge graphs, in open tooling [16] and in emerging IETF work on a knowledge-graph framework for operations [20], rests on the same premise pursued here, that operational meaning must be held as structured, machine-consumable knowledge and not as form alone. Work on giving AI systems a canonical interface to network models, in the NAIM drafts on an AI-assisted modelling format and its operations and model-context-protocol companions [17] [18] [19], approaches the same need from the modelling side. The semantic model, the lift, and the shared references developed below are complementary to these, and a thin shared reference is precisely the kind of interoperable anchor to which such knowledge graphs and interfaces could bind.

2 Where the machine cognition sits: the cognition spectrum

Before asking how meaning should be structured, it is worth asking a question that the arrival of machine cognition raises, but that the older, form-centred way of working never had to. Take the reconciliation of two systems as the illustrative case. Two systems with different roles, viewpoints, origins, and contexts must exchange information. That exchange has to be designed, and design requires cognition. So where does the cognition lie? The answer was once implicit and uniform: cognition was a human function. Engineers read the schemas, inferred the intent behind them, designed the exchange, and settled differences by hand. The data methodologies of the field grew up around that assumption. Models were made precise enough for software to exchange, and a human was trusted to supply the meaning at each end. Once some of the systems can reason for

themselves, the answer stops being uniform. That difference turns out to govern almost everything about how a reconciliation can proceed.

The axis is simple to state. If the problem at hand is the reconciliation of two systems, **how many of them are competent, interrogable machine-cognitive agents**: two, one, or none? Where the cognition sits, relative to the systems, changes how a reconciliation can be accomplished and what it costs. Two mechanics are invoked below and developed later: completing a model from a schema and its data, treated in Section 5, and the leftover that completion cannot pin down, treated in Section 6. Here the concern is only the shape of the axis and what moves along it.

With **both** systems cognitive, reconciliation is a negotiation between peers. Each can be asked, each explains itself, and each ratifies its own side of the result. The conversation between them is the mediation, and it closes and certifies its own outcome without an external step. This is the qualitatively different case. No third party proposes correspondences for someone else to approve, because the parties settle the matter themselves. Confidence is near-certain, and nothing is left over for a human to adjudicate.

With **one** system inert, a schema and data with no agent behind it, the single machine cognitive system must do all the work. It wears two hats. It represents its own side, and it stands in for the silent one. It cannot ask the inert side what a field means. It must infer that from the schema and the instances, hold each inference with a confidence, and corroborate the inference by fit, by regenerating the native records, and by cross-checking against its own observations. The confirmation that a second agent would have given is simply absent. So some meaning cannot be recovered from data alone. A vendor code with no public semantics, or a fixed label that under-determines a graded one, remains as a **residual**: a leftover of meaning that could not be settled, which the system marks as uncertain and refers onward for validation. This is the honest boundary of what one agent can recover about a silent other.

With **neither** system cognitive, the cognition must sit above both, in a third party. A mediator infers both models from their schemas and data, with no confirmation available from either. It can only propose. Its output is a set of candidate correspondences for adjudication. Its analysis is not a negotiation between the systems, because the systems are not present to negotiate. Closure must come from outside, from a human or from a downstream test that exercises the mapping.

The pattern across the axis is a gradient. As the machine cognition moves out of the systems and above them, more must be inferred without confirmation, confidence falls, and the residual grows. The burden of ratification passes outward, from the parties themselves, to an optional human review, and then to a required external one. Table 1 sets this out. The method itself does not change along the axis. The same completing, reconciling, and projecting is done throughout, and correctness on what is recovered holds. What one-sided and no-sided cognition cost is confidence, completeness, and the ability to close without human help.

	Both cognitive	One inert	Both inert
Cognition sits	in both systems	in the one live system	above both, in a third party

How the other model is obtained	disclosed by each	inferred by the live system	inferred by the mediator, for both
Confirmation	bilateral	by evidence, no confirmer	none; candidates only
Residual	none	the inferred side's opaque parts	both sides' opaque parts
Ratification	self, by both parties	optional human, for the residual	external, required
Character	the conversation is the mediation	one system mediates for a silent other	a third party proposes; it is not a party

Table 1. What changes along the cognition spectrum.

Two needs follow from this picture, and they set up the rest of the work. We would like reconciliation to be as autonomous as the available machine cognition allows, and no less reliable when that cognition is scarce. We would also like to understand what, short of full cognition on both sides, can recover some of what full cognition provides. The spectrum is the measuring stick to which the paper returns. The semantic model, the lift, and the shared references that follow are all to be judged by how well they carry a reconciliation across this axis, and especially by how much they preserve as the machine cognition recedes.

3 From data to meaning

Return, then, to the question the spectrum sharpens. What must be added to data, as delivered under a particular schema, so that a machine can comprehend it without a human standing in the gap? It helps first to fix a vocabulary, in the terms this work will use throughout. Meaning is treated as having two parts, over a common base. The first part is an **ontology**, the entities and relations a body of content is about, together with a **lexicon**, the terms by which those entities and relations are named and told apart. The second part is **pragmatics**, how the content is to be used and what its use presupposes about context. Beneath both runs **provenance**: who asserted a thing, by what method, and how firmly. A semantic model, defined in Section 4, is exactly ontology, including its lexicon, plus pragmatics, carried over that base of provenance. The phrase names those same parts each time it recurs in this paper.

These parts have a long grounding in the study of signs, which distinguishes three aspects of any representation [1]. **Syntax** is its form, the rules by which it is well constructed. **Semantics** is what its parts denote. **Pragmatics** is how it is used. The correspondence to our vocabulary is direct. Syntax is form, and a semantic model sets it aside, since the same meaning may be written in many forms. Semantics, what the parts denote, is carried by the ontology and its lexicon. Pragmatics keeps its name. Data models are strong on syntax and carry some semantics. What they say very little about, in a form a machine can act on, is pragmatics. Yet pragmatics is a large part of what a human engineer supplies when reading a model: the knowledge that a figure is a request rather than a measurement, that a route is the provider's business and not the customer's, and that an alarm during a maintenance window means something different from the same alarm at random.

An alternative conception places the same material on a ladder, the progression from data to information to knowledge and onward to wisdom [2]. Our parts line up with it directly. Data are values, carried by a data model under a particular schema. Information is data placed in relation and given reference, which is what fixing an ontology and its lexicon provides. Knowledge is information organised so that it supports inference and action, and this is reached by adding pragmatics. The semantic model, ontology and lexicon plus pragmatics, is therefore knowledge in these terms. Wisdom, the top of the ladder, is not a further document. It is cognition itself: the faculty that consumes, manipulates, and generates knowledge. On this reading a data model delivers data, the lift carries data up through information to knowledge, and the machine cognitive system supplies the wisdom that acts on it.

One further point of vocabulary will matter later. Abstraction is the level of a view. It tailors the detail to a purpose, so that raising or lowering the abstraction adds or prunes detail according to need. Abstraction is therefore a matter of ontology, tied to purpose, role, and context. Bridging a difference in abstraction is accordingly a matter of ontological reconciliation, and, as Section 6 notes, it usually carries a difference of pragmatics along with it. We note this here, lightly, and return to it where it bears on the cases.

4 The semantic model: completeness for comprehension

By a semantic model we mean what a representation needs in order to be **complete**, in the specific sense that a machine cognitive system can comprehend it and act on it without a human supplying the meaning a particular form may leave out. A data model gives form and some of the denotation. What it defers to specification, to convention, and to the engineer's own knowledge is exactly what a machine does not have. The semantic model supplies that remainder. Concretely, and as fixed in Section 3, it is an **ontology, including its lexicon, together with pragmatics**, carried over a base of provenance; in the terms of the ladder, it is knowledge. The **ontology** is the entities and relations the content is about. The **lexicon** is the terms by which those are named and disambiguated, so that a single concept under two labels, or two concepts under one label, can be told apart. The **pragmatics**, the part data models leave out, govern how the content is to be used. And **provenance** runs across all of it, recording who asserted a given fact, by what method, and how firmly.

Two things are worth separating at the outset, because they are easily bundled together and are in fact distinct operations on this object. **Completing** a representation, producing a semantic model at all so that a machine can comprehend it, is one thing. It is needed even for a single cognitive system in isolation, with nothing to reconcile against. It is simply the price of feeding a machine cognitive system. **Reconciling** two such models, making them correspond, is another thing, and it is needed when two systems must meet. The first is the subject of Section 5, the lift. The second is the subject of Section 6.

The pragmatic annotations are worth naming, because they are the substance of what completing the information means in practice. Over and above the ontology and lexicon, a semantic model records the following as first-class content:

- **use**: what a value is for, for example whether a stated capacity is a request to be satisfied or a property being reported;
- **realm**: whose decision a fact reflects, for example the boundary between what a consumer asks for and how a provider realises it;
- **state**: whether the content describes what was intended or what is operationally the case, a distinction with a long standing in datastore design [3];
- **authority**: which party is the source of truth for a given fact, and therefore whose value prevails on conflict;
- **adequacy**: how much fidelity is enough for the purpose at hand, so that a consumer knows which parts are load-bearing;
- **context**: the surrounding conditions that change what the content means, such as a maintenance window or a seasonal baseline; and
- **provenance and confidence**: the meta-band recording source, method, and trust.

Two properties of the semantic model are worth stating plainly. First, it is **complete and self-contained**. It is a portable set of information, digestible by cognition in any context. The same model serves operational reasoning within a single system and the linking of systems, and of their information spaces, through reconciliation. Because it is read as meaning, as ontology, lexicon, and pragmatics, a machine cognitive system consumes it without parsing any particular form, and the concrete forms are projections of it. Second, it is **ad hoc, not universal**. We say ad hoc for two reasons. We care about its contents, not its form. And a given model either arises from data or is generated to purpose by a system, so it is built for that purpose rather than adopted from a standing, comprehensive model of the domain. There is no universal model that all parties must accept, and standardised artifacts serve only as thin supports. It is convenient to picture the semantic model as an artifact kept “on the shelf” and drawn on directly.

5 The lift: completing a representation

The operation that produces a semantic model from a data model we call the **lift**. It is worth naming on its own, because it is needed before any second system is in view. Completing a single model is already the price of feeding a machine cognitive system, and much of the practical value of this approach is there, in completion alone.

Given a schema and a body of instances, the lift reconstructs the **ontology** the schema expresses, distils a **lexicon** for its terms, and attaches the **pragmatics** that the schema leaves to convention. It is heavier than parsing, because it recovers structure and intent rather than reading them off. Its fidelity can be checked. One regenerates the native form from the recovered model and confirms that nothing essential was lost. Where the system behind the model can be interrogated, the lift is helped by the system explaining itself. Where it cannot, the lift proceeds from the schema and the instances alone, its results are held with stated confidence, and whatever cannot be settled from data remains as the residual described in Section 2. That difference, whether a live agent stands behind the model or not, is exactly the cognition spectrum. It is why the same lift yields a self-ratifying result in one case and a referred, uncertain one in another. In both cases the lift has a definite product: a representation completed to the point where a machine can comprehend it.

6 Reconciliation: making two semantic models correspond

Reconciliation is a different operation, and it enters when two *different* semantic models must be made to correspond. There is more than one reason this need arises. Two systems may model one network as peers. A consumer and a provider may meet at a service boundary. Most generally, two systems may be built on **different modelling bases** that must nonetheless interoperate. In every case the objective is the same. There are two models, each completed by the lift of Section 5, that describe overlapping matter in incompatible ways. The task is to establish, meaning element by meaning element, how one corresponds to the other.

Reconciliation is carried out **at the level of the semantic models**. Aligning ontologies, lexicons, and pragmatics is the whole of it, because that is where the meaning lives. The product is a single reconciled semantic model, one that merges, unites, or bridges the two. Anything beyond that is a matter of generating artefacts from the reconciled model. Where two systems must actually exchange native data, translators between their data models are produced as projections of it. Those translators are outputs of reconciliation, not a second, lower level at which reconciliation is redone.

Because the translators are projections of one reconciled model, they scale well. Suppose N systems each bind once to a shared reference. The pairwise translators among them are then generated from those N bindings. Authoring a translator for every pair would instead grow with the square of N. The reconciled semantic model, anchored on a shared reference, is in this sense a hub. It turns a quadratic authoring problem into a linear binding one. Section 7 returns to this, because it is the shared reference that makes the binding cheap.

The shapes of divergence

Reconciliation is only interesting because models diverge. Since the semantic model has definite parts, its divergences can be named by the part in which they fall. A difference in the **lexicon** is a difference of naming: the same concept under different labels, or, more dangerous, different concepts under the same label, a false cognate. A difference in the **ontology** is a difference in how the world is carved: how finely it is divided, what it includes and omits, and the level of abstraction at which it is drawn. A difference in the **pragmatics** is a difference in how the content is to be used: its intended use, the authority behind it, or the context that conditions it. Two independently built models rarely share all three, so every reconciliation is in this sense cross-domain. Abstraction is worth a note of its own. Because a level of abstraction tailors detail to a purpose, a difference in abstraction usually shows up twice at once: as an ontological difference, in the detail added or pruned, and as a pragmatic one, in the purpose that motivated it. Naming the part in which a divergence falls is the first step in closing it, and misreading one part for another is where silent errors enter.

These divergences recur across a small number of operational settings, which the demonstrations of Section 8 make concrete: two peer configuration models of one network; a declarative intent refined to a concrete realisation; two systems in different domains meeting only where a service rides an underlay; and observability reconciled across an incumbent fault model and a newer one built on explicit anomaly semantics.

Verification

How a reconciliation is verified depends on the case, and it also depends on where the cognition sits. Where the two models describe the same objects, agreement can be checked by round-trip on a set of semantic invariants, such as endpoint identity, connectivity, capacity, and layer relationships, rather than on byte-level identity. Where one side refines the other, as when an intent is realised, identity is the wrong test, because refinement is lossy and under-determined. The right test there is **satisfaction**, that the chosen realisation meets every stated bound, with a feasibility judgement where a bound cannot be met. Where two systems share no common vocabulary, the test becomes a worked exchange, run through and read back to confirm that each requirement was honoured. The cognition spectrum sets how much of this can be closed-loop. In the fully cognitive case the live agents can run the worked exchange between themselves and ratify the result, a genuine closed loop. As the cognition recedes, verification leans more on invariant round-trips and on external tests, because the parties are no longer present to confirm their own side. Choosing the verification that fits the case, and the cognition available, is part of the method.

7 Shared references

The candidate answer to the spectrum's central question, what short of full cognition on both sides recovers some of what full cognition provides, is a shared reference. In the demonstrations, a thin lexical reference does most of the work. It is an identity-only anchor for each concept, giving an identifier, a preferred label and synonyms, a shallow class, a disambiguating definition, and a single canonical example. It is deliberately small, and deliberately not a model of the domain. What it buys is out of proportion to its size. Because each side binds its terms to the reference, genuine correspondences arrive as near-identity rather than as candidates to be argued. False cognates are pre-empted, because two terms that merely share a label bind to different entries. The residual work is confined to what the reference does not cover. And, as noted in Section 6, binding scales with the number of systems rather than with the number of pairs.

The lexical reference is one of a family. Different references pin different facets of meaning, and several are worth studying:

Reference	What it pins	Where it helps most
Thin lexical	what a term means	everywhere; false cognates
Instance / identity	which individual	cross-domain; observability
Unit / measurement	how much, in what terms	quantitative intent
Value-set / enumeration	discrete values and ordering	severities; discrete signals
Structural / relational	shape	granularity-heavy config
Pragmatic	use and context	intent; observability
Provenance / trust	confidence, source, method	the inert cases
Canonical scenario	meaning by shared example	verification
Invariant / constraint	what must be preserved	round-trip and satisfaction

Table 2. A family of shared references, by the facet of meaning each pins.

The reason to study references, rather than simply to use them, is a hypothesis about the cognition spectrum of Section 2. A reference reduces the cognitive work of reconciliation in every case. But its value looks likely to grow as the machine cognition becomes scarce. Where both systems are cognitive, a reference mostly saves conversation. Where one is inert, it stands in for the confirmation the missing system cannot give. Where both are inert, it may be the largest single lever on correctness, and on shrinking the residual that would otherwise fall to a human. If that is so, a reference partly substitutes for cognition. The practical recommendation, how much to invest in shared references, then depends on how much machine cognition is present. Establishing whether it is so, and by how much, is the point of the study.

8 Four demonstrations

The argument is made concrete by four worked reconciliations. Each is an operational setting in which two systems hold divergent models of one network. Each lifts the two models into semantic models, reconciles their meanings onto a shared reference anchored where a suitable one exists, closes the residual, and verifies the result by the test that fits the case. Each ends by assembling the reconciled meaning as a semantic model that a machine cognitive system could comprehend directly. Each is provided as a self-contained, interactive HTML file, named in the heading of its subsection, and they are described here in enough detail to stand on their own.

Configuration: two peer models of one transport network (configuration.html)

The first reconciles two standardised models of a single optical transport network [4] [9]. One is organised as a single multi-layer topology, the other as per-layer networks stitched together. The divergences are lexical, of granularity, and of scope. The instructive case is a false cognate. A link-termination on one side and a tunnel-termination on the other share the shape of a name but denote different things, one an end of a link and the other the head of a trail. Binding both to a thin reference, in which they map to different entries, pre-empts the error before it can be proposed. The residual is a single multi-layer node against per-layer nodes, and one identifier space against another. It is closed by derived transforms, and by each side extending itself to cover a concept the other has and it lacks. The whole is verified by round-trip on the invariants of the network.

Intent: refining a quantitative demand to a realisation (intent.html)

The second bridges a declarative intent [15] and a concrete realisation [10] [11]. The intent is expressed as quantitative expectations: a lower bound on capacity, an upper bound on latency, an availability target, and a protection requirement. The service model realises them with discrete signals, routes, and wavelengths. The mapping here is a refinement, not an equivalence. An expectation such as a capacity of at least eight units is not equal to a particular discrete signal; it is satisfied by it. The verification is therefore satisfaction rather than round-trip. Where two expectations conflict, the case shows the provider returning a best-achievable offer, and the intent's own judgement accepting a degraded value against its declared priorities, with no human in the loop. The same machinery, run in the other direction, carries operational state back up as fulfilment. It distinguishes a breach the network can remediate on its own from one that requires the consumer to accept, renegotiate, or withdraw.

Cross-domain: two bespoke systems with no common standard (cross_domain.html)

The third is the hardest of the four. Two systems sit in different domains, one provisioning transport and the other provisioning an IP service that rides it. There is no standard vocabulary on either side, and only the one seam where a service meets its underlay. With no standard to adopt, the two build a minimal shared reference between themselves. They agree each category only when both can instantiate the same canonical example, and they pin every provisioning requirement to such an example so that it cannot be misread: a line rate against a committed payload rate, a protection scheme against a stated survivability, a tier against a number. A word shared across the two systems, a “grade” that is a rate class on one side and a fibre class on the other, is exactly the trap the reference dissolves. Verification is a worked provisioning, read back to confirm that each pinned requirement was honoured.

Observability: reconciling anomaly semantics across generations (anomaly.html)

The fourth reconciles two ways of describing what is wrong with a network. The incumbent fault model bundles an alarm from an event, a state, a fixed severity, and a probable cause [5] [6]. A newer model separates these [8] [12] [13] [14]. It distinguishes an anomaly, a deviation that may be benign, from a symptom, a fault, a persistent alarm-state, a problem, a cause, and a correlated incident. The dangerous cognate here is the alarm itself. An alarm is a declared undesirable state; an anomaly is a deviation that may mean nothing; treating them as the same term is a category error. The case turns on pragmatics. The same rising measurement is a page during normal operation, benign inside a maintenance window, and a matter to watch when the detector's confidence is low. Where the incumbent model would raise a spray of uncorrelated alarms, the reconciled view resolves them into a single incident with a probable cause. This case makes the point, more sharply than the others, that in observability the meaning of a signal is carried largely by its pragmatics.

The same four settings serve to illustrate the cognition spectrum of Section 2. Take the observability case and vary which side is a machine cognitive agent. This reproduces, in turn, the fully cognitive reconciliation; the one-inert case, in which the reasoning side infers the other and carries a residual; and the both-inert case, in which a third-party mediator infers both and can only propose. The gradient of Table 1 is visible directly, as falling confidence, growing residual, and ratification passing outward to a human.

9 Goals of the proposed study

The theory and the demonstrations suggest a study, which Part II develops. Its aims are, in short:

- to find the shared references that are most effective, judged by the reconciliation work they save and the reliability they add, and that remain practical and flexible enough to be built for real cases;
- to quantify that effectiveness and its benefit, in realistic cases across the categories illustrated by the demonstrations, and across the cognition spectrum, rather than to assert it;

- to delineate the cognitive workflows that reconciliation involves, and to quantify their intensity, so that the heavy and the mechanical stages can be told apart; and
- to understand where a reference reduces the requirement for human supervision or decision, which matters most precisely where machine cognition is not present on both sides.

Part II turns these aims into questions, a method, and a plan. It asks what data and what experiments, drawn from real networks, would let the questions be answered.

References

- [1] C. W. Morris, Foundations of the Theory of Signs, University of Chicago Press, 1938.
- [2] R. L. Ackoff, "From Data to Wisdom," Journal of Applied Systems Analysis, vol. 16, 1989.
- [3] RFC 8342, Network Management Datastore Architecture (NMDA), IETF, 2018.
- [4] RFC 8795, YANG Data Model for Traffic Engineering (TE) Topologies, IETF, 2020.
- [5] RFC 8632, A YANG Data Model for Alarm Management, IETF, 2019.
- [6] RFC 5424, The Syslog Protocol, IETF, 2009.
- [7] RFC 9232, Network Telemetry Framework, IETF, 2022.
- [8] RFC 9940, Some Key Terms for Network Fault and Problem Management, IETF, 2026.
- [9] ONF TR-547, Transport API (TAPI) Reference Implementation Agreement, Open Networking Foundation.
- [10] draft-ietf-ccamp-l1csm-yang, A YANG Data Model for L1 Connectivity Service Model (L1CSM), IETF, work in progress.
- [11] draft-ietf-teas-te-service-mapping-yang, Traffic Engineering and Service Mapping YANG Model, IETF, work in progress.
- [12] draft-ietf-nmop-network-anomaly-semantic, Semantic Metadata Annotation for Network Anomaly Detection, IETF, work in progress.
- [13] draft-ietf-nmop-network-incident-yang, A YANG Data Model for Network Incident Management, IETF, work in progress.
- [14] draft-ietf-nmop-network-anomaly-architecture, A Framework for a Network Anomaly Detection Architecture, IETF, work in progress.
- [15] TM Forum TR290, Intent Common Model; IG1253, Intent Management; TMF921, Intent Management API.
- [16] B. Peters, Network Ontology Knowledge Graphs, open-source project, github.com/bradspau/Network-Ontology-Knowledge-Graphs.
- [17] draft-feng-netmod-naim, NAIM: A Canonical Data Format for AI-Assisted YANG Modeling, IETF, work in progress.
- [18] draft-feng-netconf-naim-op, NAIM Operations (companion to the NAIM data format), IETF, work in progress.
- [19] draft-feng-nmop-naim-mcp, NAIM with the Model Context Protocol (companion to the NAIM data format), IETF, work in progress.
- [20] draft-mackey-nmop-kg-for-netops, Knowledge Graph Framework for Network Operations, IETF, work in progress.